PandaLens

可视化 pandas 数据分析工作台 · 关键亮点说明

A visual pandas workbench in the browser — highlights

- 23 种 pandas 操作可视化搭建
- 13 类分析图形
- 自动生成可运行 pandas 代码
- 620 项统计与 pandas 逐项比对
- 中英双语
- 纯前端 · 数据不出浏览器

在线地址	https://2254193662lzx.github.io/pandalens/
源代码仓库	https://github.com/2254193662lzx/pandalens
视频	PandaLens-demo.mp4（4 分 49 秒，中文讲解 + 中文字幕）

数据可视化课程项目 · 2026-10-15

一句话概括

PandaLens 把 Python pandas 的数据处理能力搬进了浏览器：**用可视化方式搭建 pandas 操作流水线，用图形理解数据，并自动得到可以直接运行的 pandas 代码。** 全部计算在本地完成，支持中英双语，内容完全一致。

一、为什么是"可视化 pandas"，而不只是一个画图工具

绝大多数数据可视化工具只做最后一公里——把结果画出来。但真正花时间的分析工作其实发生在**处理阶段**：筛选、清洗、去重、派生变量、分组聚合、透视、连接、分箱、滑动窗口。这些正是 pandas 的核心，却几乎没有人把它们做成图形界面。

PandaLens 的设计出发点是：**让数据处理本身变得可见。**

- 每个操作是一个可以看见、可以展开、可以拖动的卡片；
- 每一步实时显示输出行数以及相比上一步的增减量；
- 出错只影响那一步，后面的步骤自动暂停并标红，而不是整页崩溃；
- 整条流水线可以随时停用某一步、复制某一步、调整顺序，并立刻看到结果变化。

这样做的结果是，"分析过程"本身变成了可以展示、可以讲解、可以复现的东西，而不只是一段没人看得懂的脚本。

二、23 种 pandas 操作，全部可视化搭建

操作按四类组织，覆盖日常数据分析的绝大部分动作：

类别	包含的操作
行操作	按条件筛选、排序、取前 n 行 / 后 n 行 / 随机抽样
列操作	保留列、删除列、重命名、转换数据类型、新建计算列（公式）、数值变换、文本处理、日期时间拆分、分箱离散化、移动窗口
数据清洗	删除缺失行、填充缺失值、删除重复行、异常值筛选、重置索引
聚合与关联	分组聚合、数据透视表、宽表转长表（melt）、关联另一个数据集、相关系数矩阵

三个设计细节值得单独说明：

1. **筛选条件用可视化构建器**：选列 → 选操作符（等于、介于、包含、属于集合、为正则、为空……）→ 填值，可以多条件 AND / OR 组合；也可以切换成表达式模式直接手写。
2. **表达式引擎是自己实现的**：计算列和表达式筛选使用的是一门安全的表达式语言（不是 `eval`），支持算术、比较、逻辑、三元表达式，以及 `round`、`log`、`between`、`contains`、`year`、`datediff` 等四十多个函数，输入时实时校验并提示用到了哪些列。

3. **列菜单就是一键 pandas**：在数据表里点开任意一列的菜单，除了看这一列的统计量，还能直接把"按这列筛选""用均值填充缺失""对这列分箱""转成日期类型""按这列分组聚合"变成流水线步骤——鼠标点两下就完成了一个 pandas 操作，而且立刻能看到它对数据的影响。

三、自动生成可以直接运行的 pandas 代码

这是整个项目里最能体现"参考 pandas"的一点，也是最容易被做成花架子的地方。

PandaLens 中每一个操作都在注册表里同时声明了三件事：**可视化表单、对数据的实际效果、等价的 pandas 代码。**界面、步骤描述和生成的代码因此永远不会互相矛盾。

生成的代码不是伪代码。表达式会被真正翻译成 pandas 语法：

你在计算器里写	生成的 pandas 代码
`profit / revenue`	`df.assign(margin=df["profit"] / df["revenue"])`
`round(unit_price * units, 2)`	`df.assign(x=(df["unit_price"] * df["units"]).round(2))`
`region == "East" and profit > 0`	`(df["region"] == "East") & (df["profit"] > 0)`
`if(discount > 0.1, "high", "low")`	`np.where(df["discount"] > 0.1, "high", "low")`
`contains(product, "Chair")`	`df["product"].astype(str).str.contains("Chair", case=False, na=False)`
`year(order_date)`	`pd.to_datetime(df["order_date"]).dt.year`
`coalesce(age, 0)`	`df["age"].fillna(0)`

并且是被验证过的。 项目里有一个交叉验证脚本（`tools/verify-code.js`）：它在浏览器里搭建若干条流水线，导出生成的 Python 脚本，用本机真实的 pandas 运行，再逐值比对结果。四条覆盖筛选、计算列、分组聚合、去重、分箱、透视、滑动窗口、日期拆分的流水线，**形状、列名、数值全部一致。**透视表产生的多层索引列名（`('close', 'ALPHA')`）也被显式展平成与界面一致的名字。

四、统计结果与 pandas 逐项一致（620 项比对）

可视化工具最容易被质疑的就是"你算的数对不对"。为了能回答这个问题，项目自带一个统计验证脚本（`tools/verify-stats.js`），在五个数据集（零售订单、鸢尾花、泰坦尼克号、钻石价格、空气质量）上把 620 项统计量与 pandas 对比：

- `describe()` 的 count / mean / std / min / 25% / 50% / 75% / max
- 皮尔逊相关系数矩阵
- 分组聚合的 count / mean / sum / std

- 每列缺失值数量、`value_counts()` 频次

结果：**最大相对偏差 1.2 × 10⁻¹²（浮点误差量级），零处不一致。** 分位数使用与 numpy 相同的线性插值定义，标准差默认使用样本标准差（ddof=1），偏度和峰度使用与 pandas 相同的校正公式。

五、13 类图形，每一类都为"分析"而设计

图形不是装饰，每一种都对应一个具体的数据分析问题：

图形	回答什么问题
直方图	这一列的分布是什么形状？偏了没有？有几个峰？
箱线图	不同组的中位数和离散程度差多少？谁有离群点？
散点图	两个变量有没有关系？关系是正是负、强还是弱？
折线图	指标随时间怎么变化？
柱状图	哪个类别最大？差距有多大？
饼图 / 环形图	整体构成是怎样的？
相关性热力图	哪些变量一起动？
透视热力图	两个分类维度交叉之后，指标长什么样？
缺失值地图	缺失在哪里？哪几列最严重？缺失之间有关联吗？
散点矩阵	一次看完所有两两关系
平行坐标	多个维度同时比较，哪一组取值组合属于哪个群体？
矩形树图	两层级的构成占比
雷达图	多个指标归一化后，各群体的"画像"有什么不同？

四个让图形真正"能分析"的设计：

1. **自动洞察**：每个图形下方都会生成一句基于当前数据的结论。例如散点图会给出"1132 个数据对 · 斜率 0.12 · R^2 = 0.001（关系弱，正相关）"，直方图会判断"分布右偏（存在高值长尾）"，箱线图会报出中位数、四分位距和离群点个数。
2. **点击下钻**：在相关性热力图上点任意一个色块，立刻切换到这一对变量的散点图，并叠加最小二乘拟合线。这是从"整体相关结构"到"具体一对变量"最短的路径。
3. **缺失值专题**：缺失矩阵的行按缺失多少排序（missingno 库的做法），因此数据缺口会成簇显形；另外提供按列缺失比例的条形图和缺失相关性热力图。
4. **缺失数据如实标注**：图形只使用有效值，但会明确写出"使用 1200 行中的 1132 个有效值"，大数据量抽样绘制时也会注明"统计量仍基于全部数据计算"。

六、为长时间使用而做的交互细节

- **表头分布条**：数据表每一列的表头里嵌了一个迷你分布图（数值列是直方图，分类列是 Top 5 排行条），不用点开统计面板就能看出列的形态。
- **数值热力着色**：一键把表格里的数字变成颜色深浅，快速发现极值。
- **列菜单即统计面板**：悬停任意一列即可看到计数、缺失、均值、标准差、四分位距。
- **撤销 / 重做**：所有流水线操作都可以 Ctrl+Z 撤销、Ctrl+Shift+Z 重做。
- **键盘快捷键**：数字 1–6 切换视图，`/` 聚焦搜索，`L` 切换语言，`D` 切换深色模式，`?` 打开帮助。
- **统计剖析视图**：数据规模、缺失单元格、重复行、内存占用、常量列、高基数列、强偏斜列都会主动提示，并给出 0–100 的质量评分。
- **导出**：当前视图导出 CSV / JSON，流水线导出 pandas 脚本或 JSON 定义，一键生成 Markdown 分析报告（含规模、步骤、描述统计、缺失清单、相关性排名），图形导出 PNG 与图表数据。
- **性能**：54,940 行的钻石价格数据可以直接加载、排序、分箱、分组聚合；散点图超过数千点时自动抽样绘制，但统计量始终基于全量数据。

七、中英双语：内容完全一致

双语实现方式上没有走"两套文案各自维护"的路线，而是**一条字符串对应两种语言**：

```
'table.heatmap': ['Colour scale numbers', '数值热力着色'],
```

这种结构从根上杜绝了两种语言内容不一致或漏翻译的可能。切换语言时，界面、提示、步骤描述、图形参数、自动洞察、导出报告，**连生成代码里的中文注释**都会一起切换。项目自带检查脚本，会在运行时报告任何被引用但未定义的文案键，以及在页面上残留的未翻译文本。

八、交付物与验证方式

交付内容	位置
网站	https://2254193662lzx.github.io/pandalens/
源代码	`PandaLens-source.zip`（只含网站源码：页面、样式、脚本与第三方运行库，共 15 个文件）
视频	`PandaLens-demo.mp4`（4:49，中文讲解 + 内嵌字幕；字幕文件 `PandaLens-demo.zh.srt`）
本文档	`PandaLens-亮点说明.md` / `.docx` / `.pdf`

项目包含三套自动验证脚本，任何人都可以复现结论：

```
node tools/probe.js        # 逐一遍历所有视图、10 个数据集、13 种图形、23 种操作、双语与深浅主题
node tools/verify-code.js  # 导出生成的 pandas 代码，用真实 pandas 运行并逐值比对
node tools/verify-stats.js # 620 项统计量与 pandas 比对
```

`probe.js` 在**线上地址**上同样运行通过（0 错误、0 失败请求）。

九、技术实现

纯静态前端，没有构建步骤，没有后端，没有第三方追踪：

- **ECharts** 负责图形渲染（本地化引入，不依赖 CDN）；
- **PapaParse** 解析 CSV / TSV / TXT，自动识别分隔符；
- **SheetJS** 解析 Excel；
- 自己实现的部分：DataFrame 引擎（列式存储、类型推断、缺失值处理）、统计内核、表达式语言及其 pandas 代码生成器、操作注册表、图形注册表、双语系统、表格与表单渲染。

整个项目约 7,000 行手写代码，其中数据引擎与统计部分约 1,600 行，是"数字必须算对"的部分；操作与图形注册表约 1,500 行，是"界面与生成的代码必须永远一致"的部分。

十、可以改进的方向

诚实地说，目前还有这些限制：

- 计算列的表达式中，字符串与数值混合运算不会自动加 `.astype(str)`，生成的代码在这类边界情况下需要手工调整（普通数值与比较运算已完全一致）；
- 透视表只支持单层索引与单层列，多层嵌套透视尚未实现；
- 移动窗口只支持按行顺序计算，还没有实现 pandas 中 `groupby().rolling()` 的组内窗口；
- 时间轴上的重采样（`resample`）与时间频率转换还没有做；
- 数据规模上限取决于浏览器内存，几百万行的数据不适合在浏览器里处理。